

Coding Assignment

Deadline - 5 days

Product to create - **Book summarizer and query ai agent.**

Problem statement - To create an ai agentic based book summarizer, which takes in entire books as input and gives out a 100 word summary as an output. It should also have a feature to question anything about the book.

Deliverables - a working link to the application (and github of all related code), Documents explaining the architecture, endpoint documentation (which endpoint does what)

Technologies to be used (MANDATORY) - Fastapi, Postgres, langgraph (optional), Azure AI/AWS bedrock

Minimum features requirement -

1. A RAG based agent (NO ONE SHOT/FEW SHOT PROMPTING) (or a chain/workflow/graph of multiple agents) which summarizes huge text volumes (500+ pages books) and can also effectively deal with PDF/DOC files. (Bonus points for using langgraph, prefer to use haystack/crewai/llamaindex etc.)
 - a. Use postgres's pg_vector (keep it simple) to store documents. Create your own strategy of storing documents, be it fragmentation or storing the whole book into one vector
2. Do not train any models. Use API calls to existing LLMs. You are free to use any LLM of choice. Keep in mind token usage and cost v.s. Quality.
3. An interactive UI frontend, in which we have a feature to upload PDF and ask questions about it. The design can be basic but it should be functional.
4. The user should be able to see the history of his/her other books extracted . (IMPORTANT)
5. Make each session unique to the browser window. BONUS points for creating a simple login page so that each user can see their history at any point in time
6. Modularity
 - a. There should be a separate frontend, backend, and a database (of choice). Bonus points if the frontend is in [Next.js](#), backend in python and postgresSQL as the database
 - b. Use **Any Azure Service** for any of the above. (Use a free trial of 30 days of azure for the same. <https://azure.microsoft.com/en-us/pricing/purchase-options/azure-account>)
 - c. Write your code in such a way that the functionalities can be expanded for future features.
7. Debugging
 - a. There should be observability for all aspects of the code (LLM observability for LLM related calls, logs and traces for backend and frontend)
 - b. Have error handling in the code.
 - c. Use proper HTTP codes for conveying different messages (204 for no content, 401 for unauthorized etc.)
8. UI

- a. The UI should be functional. No need to focus on the design.
- b. It should be secure. Unauthorized users should not be able to make queries, or view queries of other users. (if login page created, else no need.)